

birdclef-2026

1st Place Solution: Noisy Student Meets Distillation

Rank	1
Team	Nikita Babych
Author	Nikita Babych
Topic ID	704752
Version	Original

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704752>

Retrieved with Kaggle CLI on 2026-07-03.

Congrats and Thanks

Congratulations to all the winners! I know how much effort it takes, especially when you're competing against 4k teams and every day a new shared notebook appears that, despite being often an overfit, still disappoints you because you can't beat it with fancier, more complex approaches.

And as always, thank you to Kaggle and the hosts for letting us compete in BirdCLEF again this year, and for introducing something new to BirdCLEF lore - Validation!! Also, I have to thank the Kaggle community, who shared interesting ideas in the Discussions and Notebooks, some of which I even incorporated.

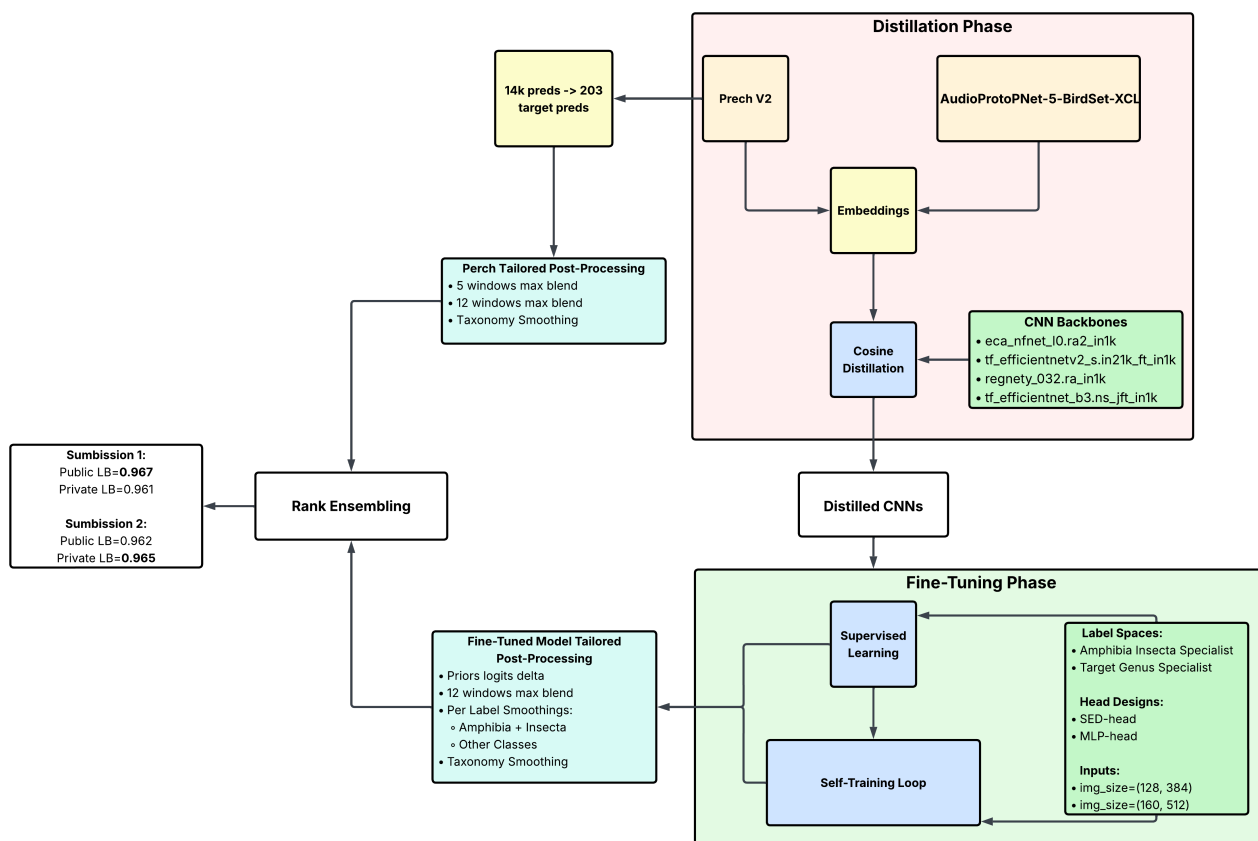
Introduction

From day one, the competition felt brutal. None of last year's ideas were producing competitive results, and I watched other participants climb the leaderboard by automating their development with coding agents while I was still coding the old-school way. With a backlog of ideas to test and a strong urge to catch up, I bought the \$100 Claude Code subscription and worked through its documentation and courses. Before long I was iterating far faster and landing solid results on the LB.

That said, I hold a strong view. SOTA agents are excellent at software development but weaker at DS/ML when it comes to novelty. So I'd argue Kaggle work splits into two parts. The first is software, the largely mechanical code-writing process, which agents automate beautifully. The second is DS/ML, where you have to generate ideas (not just reproduce what the LLM was trained on), build intuition from your results, and arrive at something original. Even the smallest such insight can be enough to be competitive. In that second part, I believe a human engineer still has the edge over fully-automated agents, since we have our "nonlinearities" that occasionally hit us with really good ideas instead of approximating the web (though maybe I'm just not exploiting the agents well enough).

Solution Overview

Before reading this write-up, I would kindly recommend getting familiar with [my last year's write-up](#), so the reader has all the context of the basis on which the current solution is built, otherwise some elements would have to be fully duplicated here, and I decided that explaining them again would be redundant.



Abbreviations

- **LSS**: labeled soundscapes
- **USS**: unlabeled soundscapes
- **PL**: pseudo-labels
- **MLP**: multilayer perceptron
- **SED**: sound event detection (attention-pooling head)
- **TTA**: test-time augmentation
- **LB**: leaderboard

TLDR

- Diverse ensemble on **5-second inputs**: SED-head CNNs, MLP-head CNNs, an Amphibia/Insecta specialist, a genus-level specialist, and Perch v2 (linear head, no fine-tuning).
- Two training phases: cosine embedding distillation (from Perch v2 [1], one model from AudioProtoPNet-5-BirdSet-XCL [2]) and full fine-tuning (supervised, then Multi-Iterative Noisy Student self-training).
- Self-training approach from my last year solution with some tweaks: normalize the injected label sum, add LSS as a separate injector, and mask non-LSS species for Site 22.
- Validation on two domain-tailored LSS splits: a Site-22 split for unseen-site generalization and a greedy split for max species coverage.
- Postprocessing tuned per pipeline (fine-tuned models vs. Perch native) mostly on the validation splits, rather than by probing the LB.

- Rank Blending between Perch v2 and my models predictions.
- AI tools: DS agent - me; software agent - Claude Code.

Data

Additional data

Any extra data I pulled from [Xeno-Canto](#) or [iNaturalist](#) for the target species made results consistently worse, so for training I used only the competition data:

- Train focal data
- LSS
- USS

The only exception was the Amphibia/Insecta specialist. For it I used the [dataset](#) of extra Xeno-Canto species I shared last year, enriched with samples and new species from iNaturalist:

- 41,127 samples
- 1,903 unique species

Focal data preparation

- I sliced audio into **5-second chunks**. Models longer than 5 seconds did not work well this year, which I explain two ways:
 - **Domain adaptation.** Our main adaptation mechanism was injecting test-domain LSS, which include very rare species appearing once in a noisy environment. Stripping context beyond the labeled 5-second chunk tells the model where to focus, and that helped.
 - **Call and labeling patterns.** The optimal duration shifts year to year with the mix of long vs. short-call species, how hard they are to separate in the domain, and how the annotators label, for example whether a year counts even a slight call overlap with a 5-second chunk. (I suspect this duration search could be automated per domain for a production system.)
- Each slice is normalized by abs-max.

LSS Preparation

I reused last year's injector concept (originally built for PL): for a fraction of each batch, mix a random LSS chunk into the background via mixup, taking the element-wise max between the focal sample's labels and the labels of the sampled 5-second LSS chunk.

With so few LSS samples, this approach was overfitting quickly even at a few injections per batch. I fixed it by normalizing the sampled LSS label sum to 0.5 instead of assigning 1 to each label, which minimizes the training signal toward those species and lets focus on the focal-only species. After that it started working pretty well.

LSS-injector parameters that were used for most trainings:

- Ratio of LSS-mixed samples per batch: **0.25 to 0.75**, depending on training phase.
- Clean (concatenated, non-mixed) LSS chunks per batch: **2**, to show a few clean test-domain samples.

- Label sum: **0.5**.
- All other preparation matches the focal data.

Validation

There is not so many LSS samples, so I did validation on the them using two complementary splits and averaged their scores:

- **Site-22 split**, move all Site 22 samples into train and validate on the rest, purely to check generalization to unseen sites.
- **Greedy split**, push as many species into validation as possible while leaving at least one sample of each species in train, to maximize per-species coverage of the metric.

I can't say that I saw a strong correlation all the time, but the signal was stable and good enough to tune the training and post-processing parameters well. For example, the training parameters I arrived at using validation turned out to be the best ones I got, even better than when I tried to readjust them specifically for the LB.

Training

Distillation

First, I tried to follow last year's solution, training models without any Perch or pretraining, because I believed in the power of in-domain self-training that can beat absolutely everything. But I got disappointed pretty quickly, seeing that my 2-iteration self-training ensemble LB=0.931 was worse than the agent-written shared notebooks using only Perch. So I accepted the rules of the club and started experimenting with Perch distillation, and I arrived at the two-phase training that worked great for me and let me get LB=0.935+ without any self-training.

1. **Distill the backbone** from the teacher's embeddings via a linear projection head using cosine loss.
2. **Disable distillation** and run standard end-to-end fine-tuning at a low LR on the distilled model (for mlp model I enabled distillation active during fine-tuning).

I trained every model this way, distilling the backbone first, then fine-tuning with whatever PL or specialist setup applied, swapping Perch for AudioProtoPNet-5-BirdSet-XCL on one model for diversity.

One detail should be noted: even when reusing the same backbone for different head/label designs, I **re-distilled backbone each time**. The distillation loss stays non-zero, which adds slight diversity between models. For example, ensembling two otherwise-identical models distilled with different seeds gave a noticable boost.

For the distillation loss I settled on **cosine** because it converged faster and better than unnormalized MSE.

Distillation phase

All models had very similar optimal training parameters, so I trained with the following:

Parameter	Value
Epochs	11

Parameter	Value
LR	5e-4
Loss	cosine
Target	teacher embeddings, no processing
Scheduler	one-cycle cosine
Data	focal train + soundscapes
Mixup	p = 0.5
Optimizer	AdamW
Batch size	64

Fine-tuning phase

In this downstream phase I lowered the LR to 2e-4 and varied epochs a bit between models; nothing else changed between supervised fine-tuning and self-training.

Parameter	Value
Epochs	from 8 to 15 (40 for Amphibia and Insecta specialist)
LR	2e-4
Loss	CE
Scheduler	one-cycle cosine
Data	focal train + LSS + pseudo-labeled USS (if enabled)
Target	normalized species/genus labels
Mixup	p = 0.5
Optimizer	AdamW
Batch size	64

Self-training: adapting last year's recipe

The last year's self-training recipe worked well for me when I trained model without any distillation and their scores were under LB=0.925. But switching to fine-tuning of the pre-distilled models any PL made results **much worse** than using no PL at all. After some search, I found the cause: distillation + LSS already give such a strong base that adding even slightly noisy PL even after power transform is fatal, they pull the entire signal toward themselves.

The first enabler was to **cap the PL label sum below the focal label sum** (same mechanism as the LSS injector, a bit different motivation). This keeps the test-domain background, adds a faint signal toward the species likely present there, and avoids overfitting to their noise. Two considerations drove it:

- LSS already provides real ground truth for some species, so noisier PL can only hurt those (pseudo labels for species from LSS were very noisy).
- Rare species not covered by LSS still need most of their signal from focal data, where PL is unreliable, so that signal must be suppressed (since I showed LSS in each batch, non-LSS species got definitely underfitted, so their values were much lower than LSS ones and usually were noise).

The second enabler was **injecting LSS and pseudo-labeled USS in the same batch** to offset PL noise on species that LSS already teaches well. Key nuance: the two injectors **must not overlap**. Take a fixed number of focal samples per batch and inject LSS into some and pseudo-labeled USS into others, never both into the same sample.

And the last thing that helped improve my results a bit (I'm talking about ~ 0.002) was masking all species for Site 22 that didn't appear in the LSS, following the prior that we have enough Site 22 samples to assume non-LSS species are rare there. Surprisingly, that worked.

Other elements from the last year's recipe were also important like power transform and sampling according max label sums.

During self-training both injectors run together, on separate focal samples:

Parameter	LSS injector	PL injector
Injection ratio per batch	0.1875	0.75
Clean concats per batch	2	-
Label sum cap	0.5	0.75

With that setup the last year's magic came again, but only for 2 iterations.

Below are results that I tracked submitting two seed eca_nfnet_l0.ra2_in1k SED models with smoothing:

Pseudo Iteration	LB score
1-stage only supervised learning	0.935
1-pseudo iteration	0.946
2-pseudo iteration	0.950
3-pseudo iteration	0.949

Models

I could not get competitive results without ensembling models with **different heads, label spaces, and inputs**, on top of different CNN families. The reason: distillation + self-training produce highly correlated models, so the only way to push further was to add non-linearity through design diversity. The boost wasn't huge, but it was enough to reach 1st on the public LB and hold it on the private.

Of all the models, the one I should explain more is the genus-level model. The thing is that a lot of species (rare amphibians specifically) couldn't be distinguished in overpopulated soundscapes, so sometimes it was enough to get the main genus signal, which is usually similar between species, to provide a very beneficial enrichment toward those otherwise unrecognizable species predictions. So I trained the model to predict genus instead of species by taking the maximum of all labels related to the same genus during data preparation (including pseudo-labels), and I blended its predictions spreading the same value to all species from the same genus, and seeing a very good boost of ~0.001-0.002 when I was stuck at LB=0.96+.

Final ensemble:

Backbone	Head	Label space	Distilled from	Pseudo iters	Mel input
regnety_032.ra_in1k	SED	target	Perch v2	2	128×384
regnety_032.ra_in1k	SED	target	Perch v2	3	128×384
eca_nfnet_l0.ra2_in1k	SED	target	Perch v2	2	128×384
eca_nfnet_l0.ra2_in1k	SED	target	AudioProtoPNet-5-BirdSet-XCL	2	128×384
tf_efficientnetv2_s.in21k_ft_in1k	SED	target	Perch v2	1	160×512
tf_efficientnetv2_s.in21k_ft_in1k	SED	target	Perch v2	2	160×512
tf_efficientnet_b3.ns_jft_in1k	SED	extended (Amphibia + Insecta)	Perch v2	supervised only	128×384
eca_nfnet_l0.ra2_in1k	MLP	target	Perch v2	2	128×384
eca_nfnet_l0.ra2_in1k	SED	genus-level	Perch v2	2	128×384
Perch v2 native	Linear	203 overlapping targets	-	-	native frontend

Mel Spectrogram parameters

Mel inputs share 32 kHz audio, 5-second clips, n_fft 2048, f_min 0, f_max 16000, power 1 (except Perch v2):

- 128×384 (n_mels 128, time 384): hop ≈ 417. Used by all eca and regnety models, plus the b3 specialist.
- 160×512 (n_mels 160, time 512): hop ≈ 313. Used by the tf_efficientnetv2_s models.

Postprocessing

The chains are tuned **separately for the fine-tuned models and for Perch v2's native predictions**, since the two have different calibration, and both were tuned on the validation splits above rather than by iterating submissions.

Fine-tuned models

1. **Site priors from LSS**, site-only combination, $\lambda = 0.5$, applied in logit space to boost species known to occur at the site - an idea taken from the shared notebooks.
2. **Max sample/window boost**, boost each chunk toward the per-file max probability for that class ($\alpha = 0.20$).
3. **Label smoothing**, temporal Gaussian smoothing with a **class-conditional kernel**: a flatter `[0.33, 0.33, 0.33]` kernel for Amphibia/Insecta and a sharper `[0.2, 0.6, 0.2]` kernel for the other labels.
4. **Genus/class taxonomy smoothing**, smooth species from the same genus ($\alpha = 0.15$) and from the same class ($\alpha = 0.05$) - an idea taken from the shared notebooks.
5. **Delta TTA**, blend framewise predictions max shifting pooling window by 2 frames (only for SED).

Perch v2 (native predictions)

1. **Window-blend smoothing**, blend of a **5-chunks-window** ($\alpha = 0.7$) and a **12-chunks-window** ($\alpha = 0.05$) max values over time.
2. **Genus/class taxonomy smoothing**, smooth species from the same genus ($\alpha = 0.15$) and from the same class ($\alpha = 0.05$)

Inference & Ensembling

Surprisingly, despite Perch's mediocre score compared to the fine-tuned models (~0.9 on the species that overlap with the LSS), its native predictions were essential to the ensemble. While experimenting with blends and submitting to the LB, I noticed that most of the Perch boost came from the non-LSS species. Those species probably scored close to Perch's, since we had no test-domain ground truth for them, so Perch did a lot of the work in identifying them.

Final predictions are a **weighted blend of the fine-tuned ensemble and Perch v2's native linear-head predictions**. Because the two come from different pipelines with different score distributions, I **rank-transform each class column before blending** rather than averaging raw probabilities.

- Fine-tuned ensemble (SED + MLP + specialists): 0.8
- Perch v2 native: 0.2

Restricted-head models (the genus and Amphibia/Insecta specialists) predict over their narrower label space and are **scattered back to the full 234-class width**, with a mask, before entering the blend.

References

1. van Merriënboer, B., Dumoulin, V., Hamer, J., Harrell, L., Burns, A., & Denton, T. (2025). Perch 2.0: The Bittern Lesson for Bioacoustics. arXiv. <https://doi.org/10.48550/arXiv.2508.04665>
2. Heinrich, R., Rauch, L., Sick, B., & Scholz, C. (2025). AudioProtoPNet: An interpretable deep learning model for bird sound classification. Ecological Informatics, 87, 103081. <https://doi.org/10.1016/j.ecoinf.2025.103081>
3. Nikita Babych. (2025). 1st Place Solution: Multi-Iterative Noisy Student Is All You Need. Kaggle. <https://doi.org/10.34740/KAGGLE/W/12619>

Resources

- Perch v2 model weights (Kaggle): <https://www.kaggle.com/models/google/bird-vocalization-classifier>
- AudioProtoPNet-5-BirdSet-XCL (Hugging Face): <https://huggingface.co/DBD-research-group/AudioProtoPNet-5-BirdSet-XCL>
- Inference Notebook: <https://www.kaggle.com/code/nikitababich/birdclef2026-1st-place-inference>
- Best ensemble models: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-models>
- Inference source code: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-src-inference>

Supplemental Q&A

Cody_Null • 2026-06-06T05:54:52.997000

Back to back first place performances is absolutely unreal, congratulations! Curious if you plan to share your data processing / training pipelines as I would love to see more about where I went astray. I didn't spend much time in the competition this year but it seems like in my custom training pipeline I had similar ideas in many portions so I'm curious how it all tied together in training

Nikita Babych • 2026-06-07T19:58:59.177000

Thank you. I've just shared the inference source code, which already includes most elements of my solution (architectures, pre-processing, post-processing, etc.). I hope I'll be able to share the rest by creating a dedicated repo.

Inference src: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-src-inference>